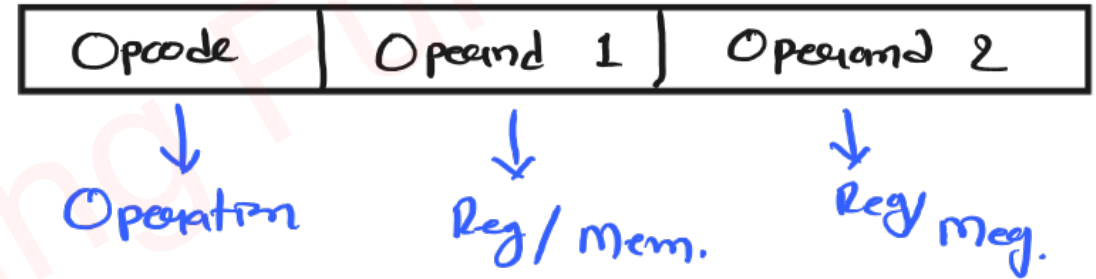
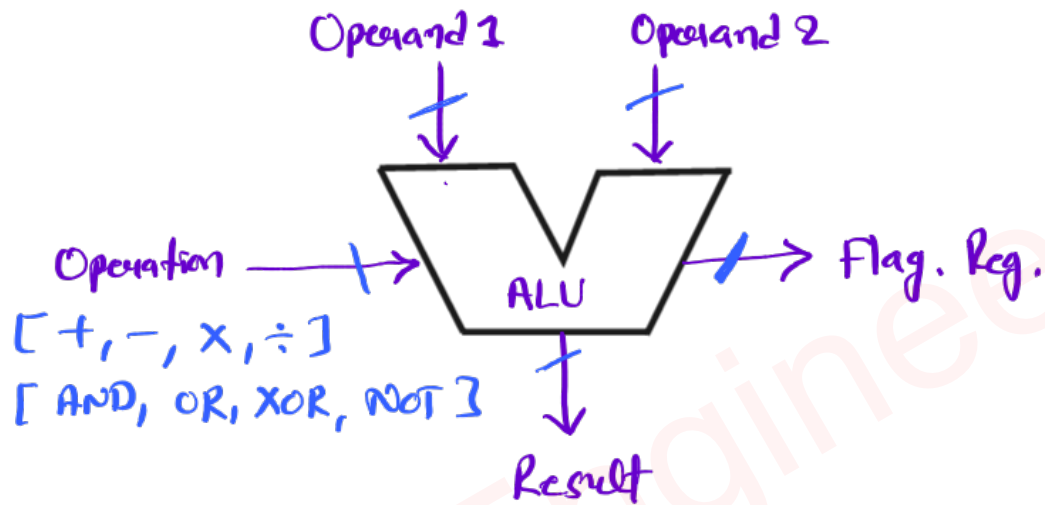


Instructions Formats & ALU in COA

- ❑ Instruction formats are based on how ALU executes instructions.
- ❑ ALU takes operands as inputs to perform execution of instructions.
- ❑ Operands may be there in Registers or in Memory, based on that instructions formats are there with different computers.



Stack CPU in COA

- ❑ In Stack CPU, ALU operations are performed on stack data only.
- ❑ So, both operands of ALU are always there on stack only.
- ❑ Stack works as per Last In First Out {LIFO}.
- ❑ Default address of operand becomes Top of Stack.
- ❑ So here, No address is required in instruction format with stack CPU.
- ❑ So instruction Format includes only Opcode.

Opcode

- ❑ In Stack Insertion and deletion takes place at the same time.

Example $R_4 = (R_1 + R_2) * R_3$

Instⁿ

Push R_1

Push R_2

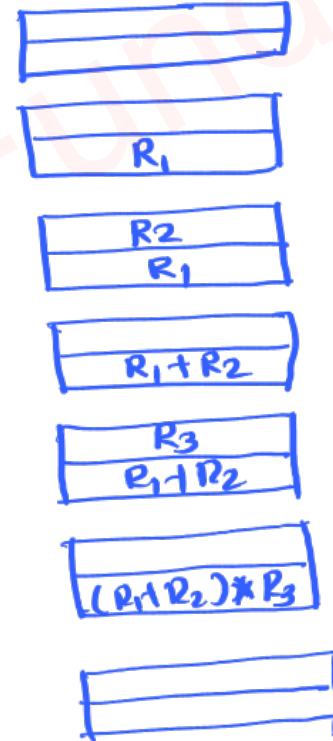
Add

Push R_3

MUL

POP R_4

Stack

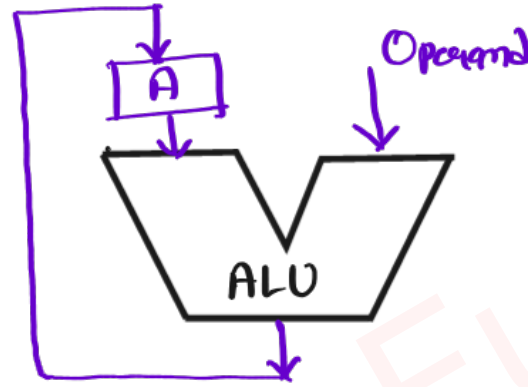


Accumulator CPU in COA

- ❑ In Accumulator CPU, one operand always stored in Accumulator.
- ❑ So, second operand may be any general purpose register or Memory.
- ❑ After arithmetic or Logical operation answer will be stored in Accumulator.
- ❑ As one operand is Accumulator, in instruction we need address of second operand only.
- ❑ So basic instruction format is

Opcode	Operand
--------	---------

↓
Operation



Example $R_4 = (R_1 + R_2) * R_3$

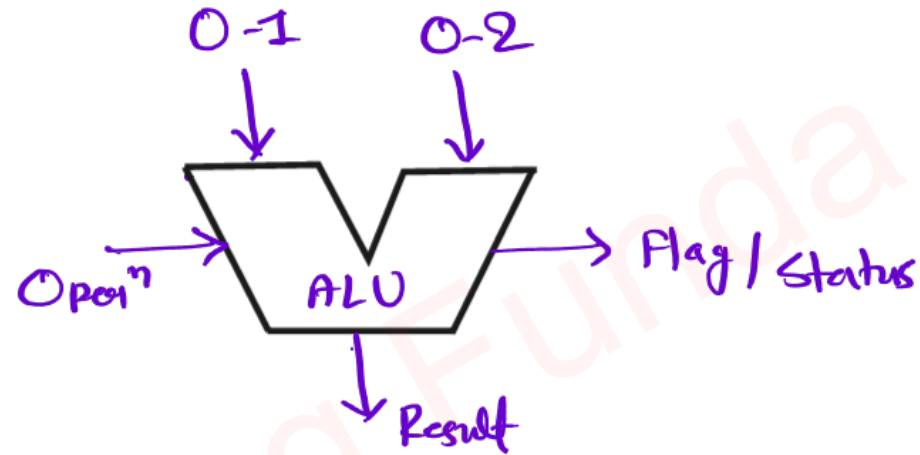
mov A, R ₁	; A ← R ₁
ADD R ₂	; A ← R ₂ + A
MUL R ₃	; A ← A * R ₃
mov R ₄ , A	; R ₄ ← A

General Purpose CPU in COA

- ❑ In this type of CPU, operands may be there in register or in memory.
- ❑ If operands are there in registers then CPU will be having more registers.
- ❑ If operands are there in memory then CPU will be having less registers.
- ❑ So, instruction format is:

Opcode	Operand1	Operand2
--------	----------	----------

↓
Operⁿ.

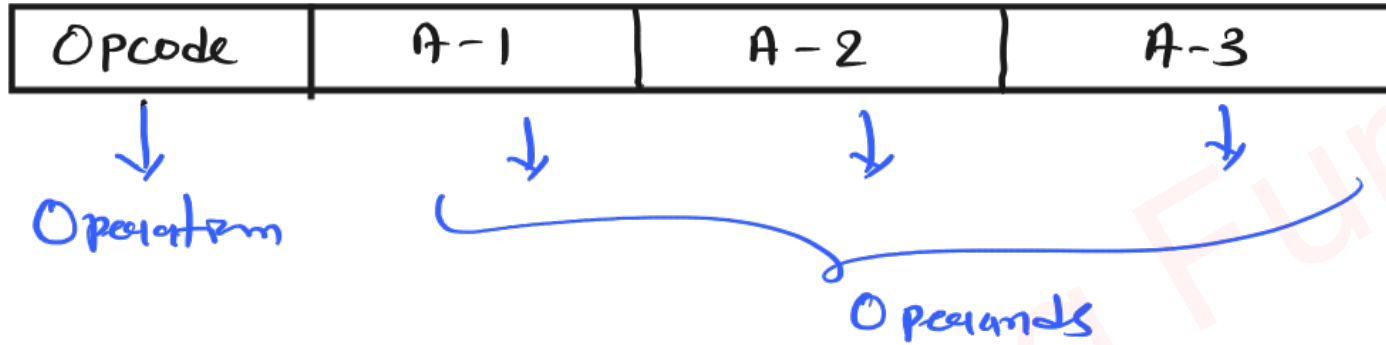


Example $R_4 = (R_1 + R_2) * R_3$

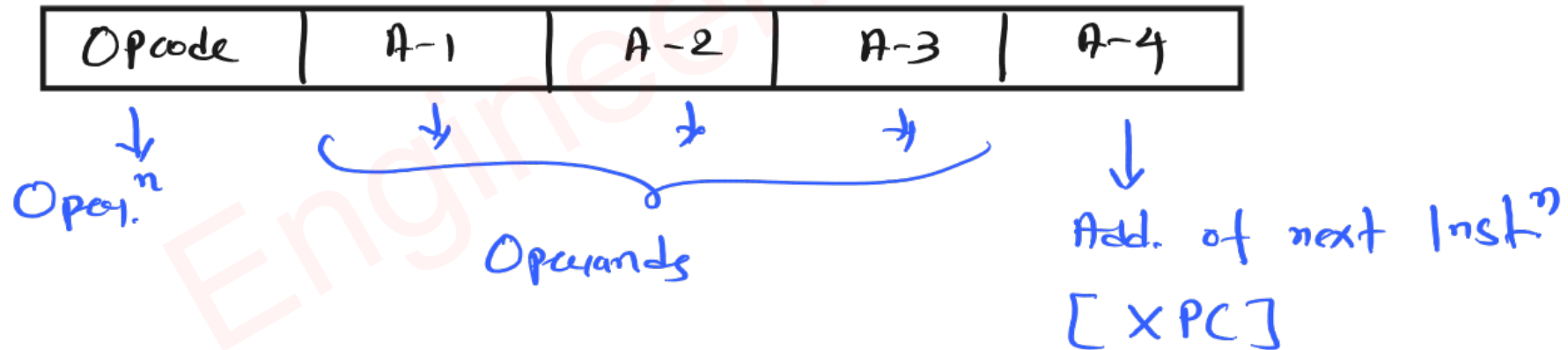
→ ADD R₁, R₂ ; R₁ ← R₁ + R₂
MUL R₁, R₃ ; R₁ ← R₁ * R₃
MOV R₄, R₁ ; R₄ ← R₁

Instructions Formats in COA

- 3 Add. Instⁿ



- 4 Add. Instⁿ



Examples on Instruction Formats in COA

- ❑ **GATE CS** – A Processor has 512 distinct instructions and 80 general purpose registers. A 24 bits instruction word has an opcode, one register operand and a memory operand. How many bits are reserved for the memory operand field.

- A. 6
- B. 7
- ✓ C. 8
- D. 9

→ Instructions = 512

Registers = 80

Size of Inst.ⁿ = 24 bits.

Opcode	Reg. Operand	Mem. Operand.
9 bits	7 bits	?

$$\begin{aligned}\rightarrow \text{Mem. Operand} &= 24 - 9 - 7 \\ &= 8 \text{ bits}\end{aligned}$$

❑ **GATE 2016 CS** – A Processor has 40 distinct instructions and 24 general purpose registers. A 32 bits instruction word has an opcode, two register operands and an immediate operand. The numbers of bits available for the immediate operand is

- A. 16
- B. 17
- C. 18
- D. 19

→ Instructions = 40

Registers = 24

Size of Inst.ⁿ = 32 bits.

Opcode	R.O. - 1	R.O. - 2	Immediate Operand.
6 bits	5 bits	5 bits	

→ Immediate Operand = $32 - 6 - 5 - 5$
= 16 bits.

❑ **GATE CS** – A Machine has 24 bits instruction format. It has 32 registers and each of which is 32 bits long. It needs to support 49 instructions. Each instruction has two register operands and one immediate operand. If the immediate operand is signed integer, the minimum value of immediate operand is

A. -64

→ Size of Instⁿ = 24 bits.

✓ B. -128

Reg. = 32

C. -256

Inst.ⁿ = 49

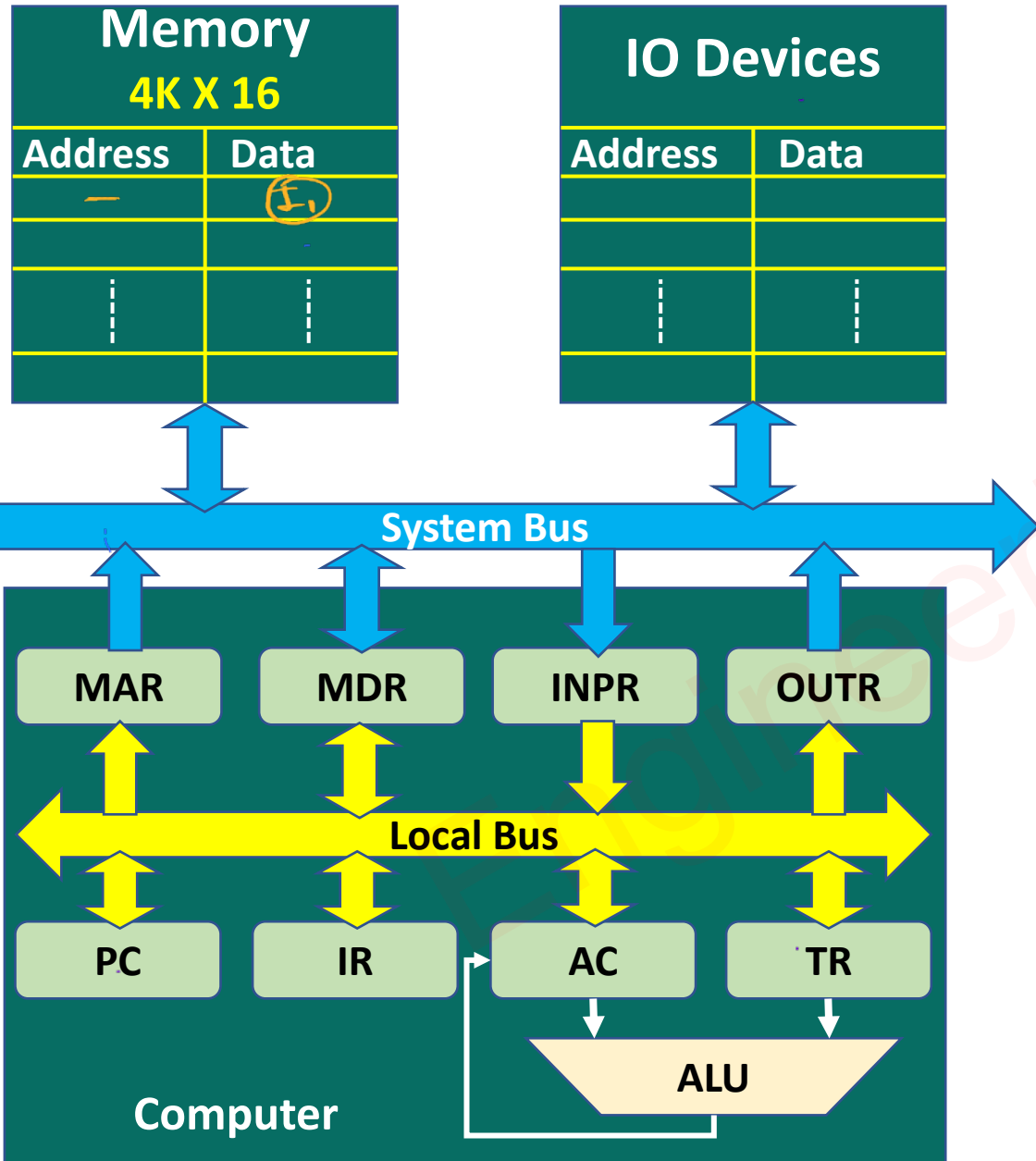
D. -32

Opcode	R.O. 1	R.O. 2	Immediate operand
6 bits	5 bits	5 bits.	

→ Immediate operand = $24 - 6 - 5 - 5 = 8$ bits.

$$\begin{array}{ccc} & \nwarrow & \\ -2^{n-1} & \text{to} & + (2^{n-1} - 1) \\ \hline -128 & \text{to} & +127 \end{array}$$

Instruction Cycle in COA



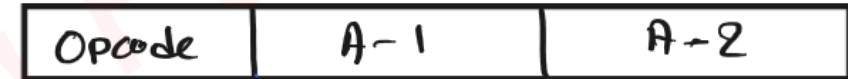
1) Fetch Instruction

$MAR \leftarrow PC$

$IR \leftarrow m[MAR]$

$PC \leftarrow PC + 1$

2) Decode Instruction



↓
Operation

Operand.

3) Decode Add of Operand.

4) Fetch Operand.

5) Execute Inst.ⁿ

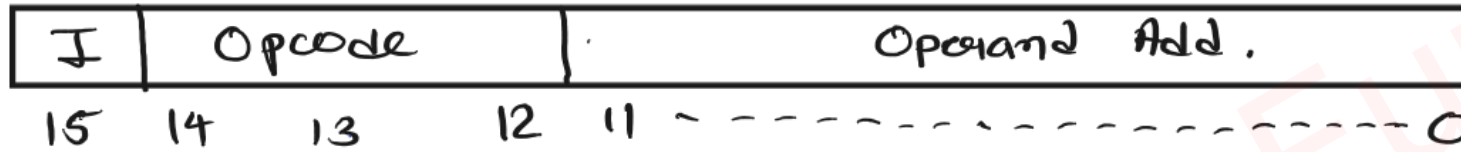
6) Write back Oper.ⁿ

Timings & Flowchart Instruction Cycle in COA

$T_0: MAR \leftarrow PC$

$T_1: IR \leftarrow m[MAR], PC \leftarrow PC + 1$

} Inst.ⁿ Fetch.



$T_2: \underbrace{D_7 \dots D_0}_{\text{Operation}} \leftarrow IR(14-12), \underbrace{MAR \leftarrow IR(11-0)}_{\text{Operand Add.}}, \underbrace{I \leftarrow IR(15)}_{\text{mode}}.$

$D_7 = 0$, then [Memory]

$I = 1$, Indirect

$I = 0$, direct

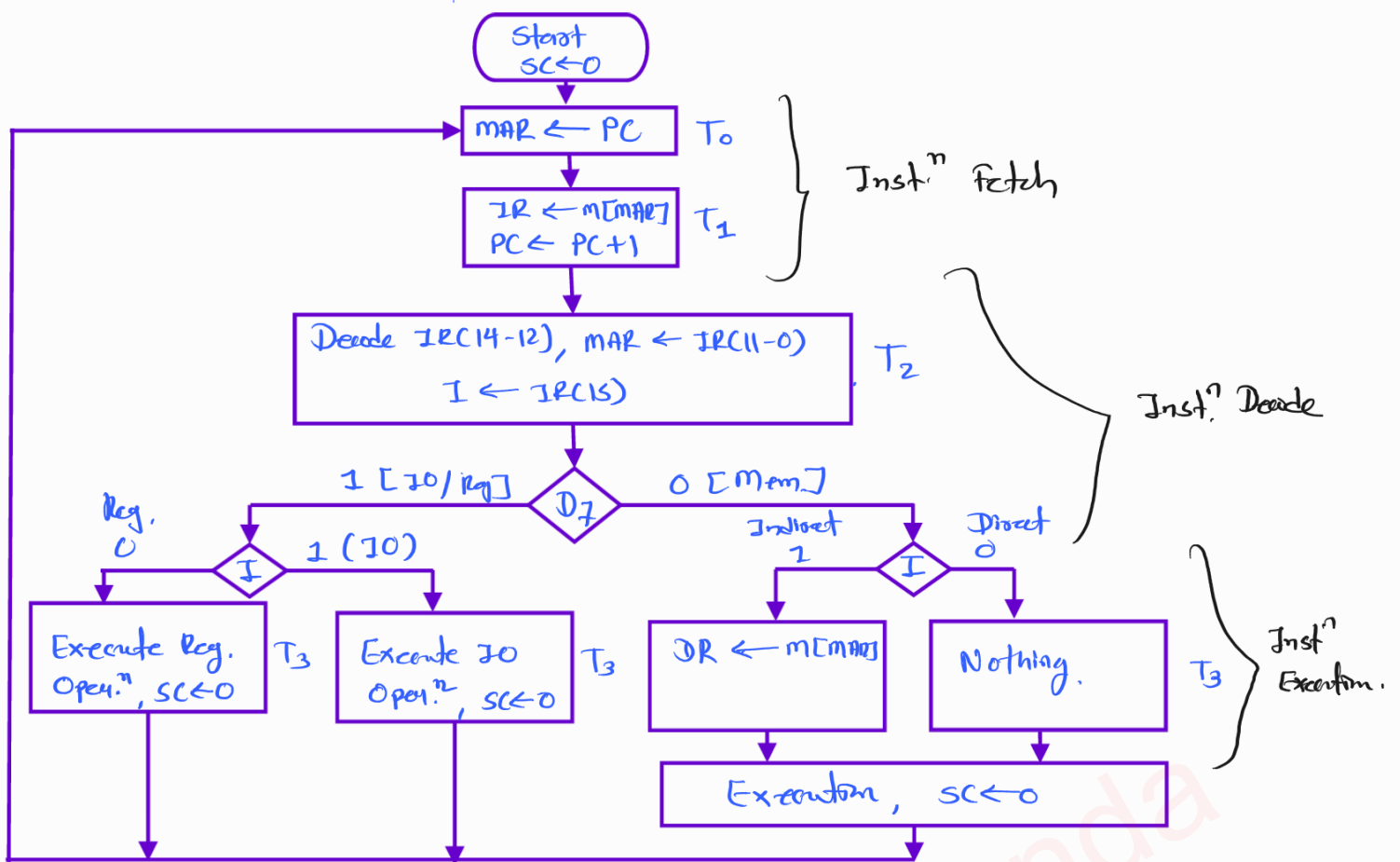
$D_7 = 1$, then [IO/Reg.]

$I = 1$, IO Oper.ⁿ

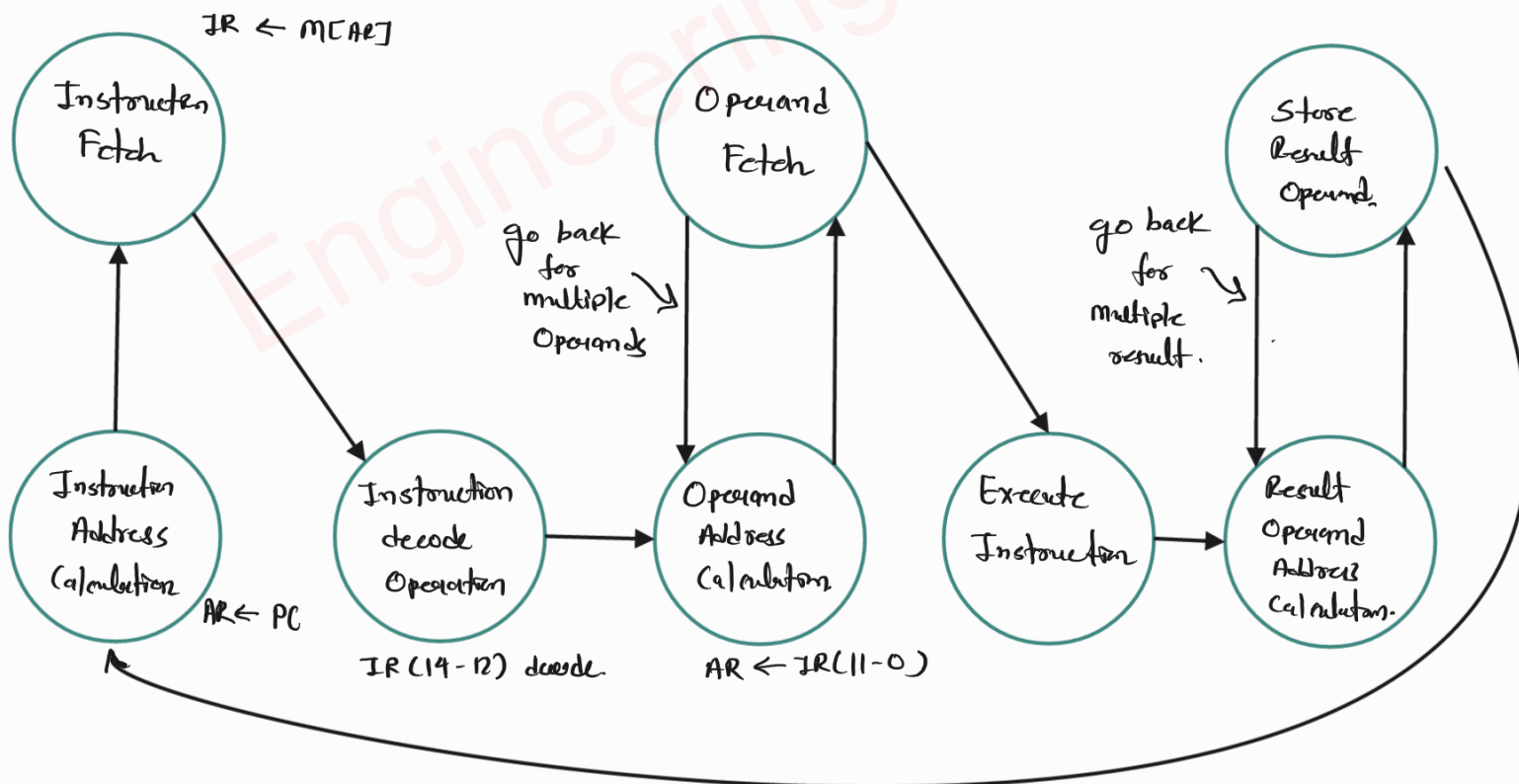
$I = 0$, Reg.

} Inst.ⁿ decode.

$T_3: \text{Execute}$ } Inst.ⁿ Execution.



Instruction Cycle State Transition Diagram in COA



Types of Instructions in COA

Data Transfer

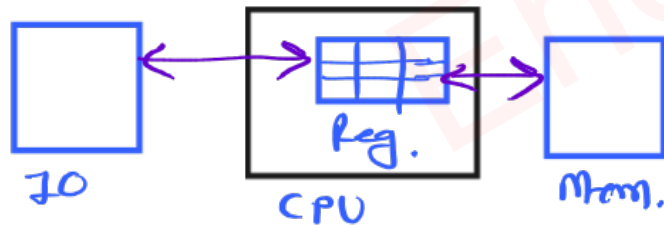
- MOV
- LD
- ST
- IN
- OUT
- PUSH
- POP

Data Manipulation

- $+$, $-$, $\times$, $\div$
- AND, OR, XOR, NOT
- Shift, rotate.

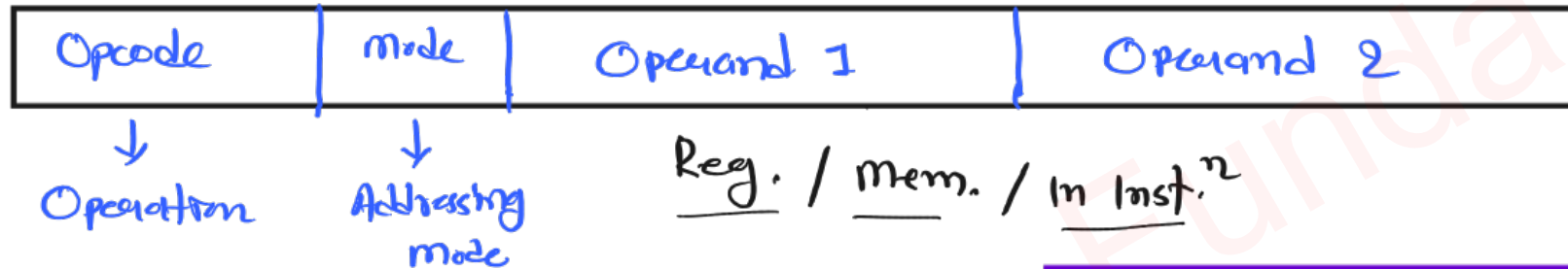
Program Control

- Jmp
- CALL
- Conditional Inst.ⁿ



Addressing Modes in COA

→ Addressing mode explains how operands are given instruction.



R1	R2
R3	R4
PC	SP
SI	DI

Registers

CPU

Address	Data
<u>1000 H</u>	→ 2000 H.
2000 H	→ (20 H)
Memory	

} Prog.

} data.

1] Immediate Addressing mode. {#}

→ Operand is given in instruction.

MOV R₁, #50H ; $R_1 \leftarrow 50H$
ADD R₁, #50H ; $R_1 \leftarrow R_1 + 50H$.

2] Implied/Implicit Addressing mode.

→ Operand is implied in instruction.

STC ; Set Carry.
CMC ; Complement Carry

3] Register Addressing mode.

→ Operands are given by registers only.

MOV R₁, R₂ ; $R_1 \leftarrow R_2$
ADD R₁, R₂ ; $R_1 \leftarrow R_1 + R_2$

4] Direct Addressing mode. []

→ Memory Address of Operand will be given in instruction

MOV R₁, [1000H] ; $R_1 \leftarrow [1000H]$
ADD R₁, [2050H] ; $R_1 \leftarrow R_1 + [2050H]$

5] Indirect Addressing mode @, (), []

→ Address of effective address of operand will be given in instruction.

LOAD R₁, @1000H

6] Register Indirect mode.

→ Address of effective address is given by register

MOV R₁, [R₂] ; $R_1 \leftarrow [R_2]$
ADD R₁, [R₃] ; $R_1 \leftarrow R_1 + [R_3]$

7] Base Indexed addressing mode.

→ Address of operand is given by Base reg. + Index reg. (Value).

MOV R₁, [R₁ + R₂]
 ↓ ↓
 Base Index.
MOV R₁, [R₁ + 20H]
 ↓ ↓
 Base Index.

8] Autoincrement - Auto decrement Addressing mode. {+, -, ++, --, !}

→ Address of Operand is given by register & that register can be incremented or decremented.

LD R₁, [R₂]+ ; R₁ ← [R₂] , R₂ ← R₂ + 1

9) PC - Relative mode

→ Program Control will get transferred by Offset.
{ intra Segment }

$$PC \leftarrow PC + \underline{\text{Offset}}$$

→ 1000H - SJMP (20H)
1002H x ← PC = 1002H.

$$PC_{\text{new}} = 1002 + 20 = 1022H.$$

10) Base register addressing mode.

→ Program Control will get transferred by base reg. + Offset.

$$PC \leftarrow \text{base reg.} + \text{Offset}.$$

{ intra segment }

Example on Addressing Modes in COA

□ **GATE CS** – Consider a four bytes instruction present in the memory stored at address of 100. Instruction have 16 bits for opcode + mode bits and 16 bits for operand. Find the effective address and value of operand for all the addressing modes. {Note: Only one register R = 200}

Address	Data	Opcode + Mode bits	Operand	Mode	EA	Operand
→ 100	xxxx			Immediate	101	300
101	Add/Data = 300			Direct	300	400
→ 102		PC = 100 102	Program Counter	Indirect	400	70
...	...			Register	—	200
199	55	R = 200	General Register	Register Indirect	200	65
200	65			Auto Dec Pre	199	55
...	...			Indexed	300 + 100	70
300	400	XR = 100	Indexed Register	PC Relative	102 + 300	—
...	...					
400	70					
401						
402	Target Inst.					

$$P_{\text{new}} \leftarrow PC + \text{Offset}$$

Examples on Addressing Modes in COA

❑ **GATE 2011 CS** – Consider a hypothetical processor with an instruction of type LW R1, 20(R2) which during execution reads a 32-bit word from memory and stores it in a 32-bit register R1. The effective address of the memory location is obtained by the addition of a constant 20 and the contents of register R2. Which of the following best reflects the addressing mode implemented by this instruction for operand in memory?

- A. Immediate Addressing
- B. Register Addressing
- C. Register Indirect Scaled Addressing
- ✓ D. Base Indexed Addressing

$$R_1 \leftarrow [\underbrace{20}_{\text{Index}} + \underbrace{R2}_{\text{Base}}]$$

❑ **GATE 2008 CS** – Which of the following is/are true of the auto increment addressing mode?

- I. It is useful in creating self relocating code.
- II. If it is included in an instruction set architecture, then an additional ALU is required for effective address calculation.
- ✓ III. The amount of increment depends on the size of the data item accessed.

A. I Only

B. II Only

✓ C. III Only

D. II and III Only

Engineering Funda

❑ **GATE 2005 CS** – Match each of the high level language statements given on the left hand side with the most natural addressing mode from those listed on the right hand side.

1. $A[1] = B[j]$

✓₃ a. Indirect Addressing

2. While [$*\underline{A}++$]

✓₁ b. Indexed Addressing

3. $\text{int temp} = *x$

✓₂ c. Autoincrement

A. (1, c), (2, b), (3, a)

B. (1, a), (2, c), (3, b)

✓ C. (1, b), (2, c), (3, a)

D. (1, a), (2, b), (3, c)

$[\underline{A} + \underline{1}] \leftarrow [\underline{B} + \underline{j}]$

$\text{Temp} \leftarrow [x]$

❑ **GATE 2000 CS** – The most appropriate matching for the following pairs

b 1. Indirect Addressing

a. Loops

c 2. Immediate Addressing

b. Pointers

a 3. Auto Decrement Addressing

c. Constants

A. (1, c), (2, b), (3, a)

B. (1, a), (2, c), (3, b)

✓ C. (1, b), (2, c), (3, a)

D. (1, c), (2, a), (3, 2)

Engineering Funda

Examples on Addressing Modes in COA

☐ **GATE 2006 CS** – Which of the following statements about relative addressing mode is FALSE?

- A. It enables reduced instruction size ✓
- B. It allows indexing of array elements with same instruction ✓
- C. It enables easy relocation of data ✓
- D. It enables faster address calculations than absolute addressing ✗

☐ **GATE 1999 CS** – A certain processor supports only the immediate and the direct addressing modes. Which of the following programming language features cannot be implemented on this processor? a) Pointers b) Arrays c) Records d) Recursive procedures with local variables

- A. Only a
- B. a and b
- C. a and d
- ✓ D. All

❑ **GATE 2018 CS** – Consider the following instruction sequence in a RISC machine :

Offset →

Instruction No	Instruction	Instruction Size
i: 1000	Add R1, R2, R4	4
i+1: 1004	Sub R4, R5, R6	3
i+2: 1007	Cmp R0, R8, R9	5
i+3: 1012	Mul R1, R9, R10	1
i+4: 1013	Beg R0, Offset ←	3

→ PC = 1016

❑ Conditional and unconditional branch instructions use PC- relative addressing mode with Offset specified in bytes to the target location of the branch instruction. Further the Offset is always with respect to the address of the next instruction in the program sequence. If the target of the branch instruction is i, then the decimal value of the Offset is -16

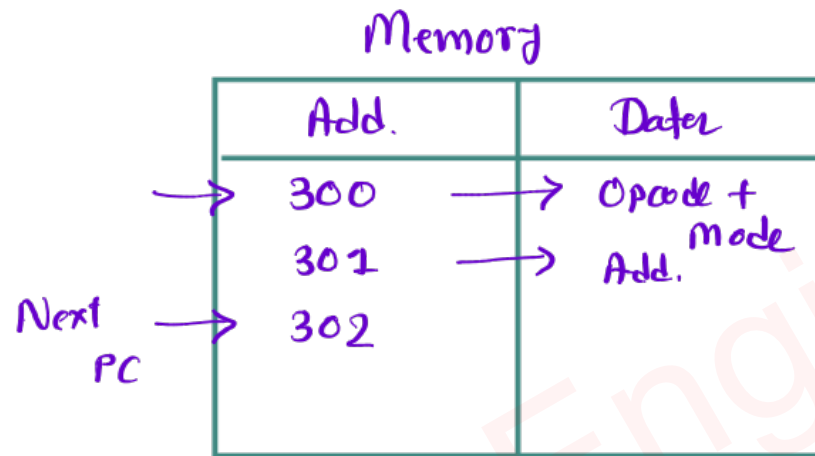
- ✓ A. -16
B. 18
C. 1000
D. None of above

$$\begin{aligned}\Rightarrow PC_{\text{new}} &= PC + \text{offset} \\ \Rightarrow 1000 &= 1016 + \text{offset} \\ \Rightarrow \text{offset} &= 1000 - 1016 \\ &= -16\end{aligned}$$

Examples on Addressing Modes in COA

- ❑ **GATE CS** – An instruction is stored at location 300 with its address field at location 301. The address field has the value 250. A processor register R1 contains the number 200. Evaluate the effective address, if addressing mode is:

- A. Direct = 250
B. Relative = $250 + 302 = 552$
C. Register Indirect = 200



$$R_1 = 200$$

❑ **GATE CS** – A relative branch mode type instruction is stored in memory at address 300. The branch is made to an address 450.

A. What should be the value of relative address field of the instruction?

B. Determine the value of PC before instruction fetch, after the fetch and after the execution phase?

Memory

Addr	Data
300	I
301	
450	

PC →

$$\Rightarrow PC_{\text{new}} = PC_{\text{old}} + \text{offset}$$

$$\Rightarrow 450 = 301 + \text{offset}$$

$$\Rightarrow \text{offset} = 149$$

→ before Fetch = 300

After Fetch = 301

After Execution = 450.

- ❑ **GATE CS** – Assume that there are 5 instructions in the loop body, starting memory locations are 3000, 3004, 3008, 3012 and 3016, respectively. Assume each instruction takes 4 bytes of memory and memory is byte addressable. Calculate the offset value needed to return to the starting of loop.

Memory

Add.	Data
→ 3000 →	I ₁
3004 →	I ₂
3008 →	I ₃
3012 →	I ₄
<u>3016 →</u>	<u>I₅</u>
PC → 3020	

$$\Rightarrow PC_{\text{new}} = PC_{\text{old}} + \text{offset}$$

$$\Rightarrow 3000 = 3020 + \text{offset}$$

$$\Rightarrow \text{offset} = -20$$

- ❑ **GATE CS** – In a certain processor, a 4 byte jump instruction is encountered at memory address 3055, the jump instruction is PC relative mode with offset of -15. Determine the target branch address.

3055 → I ✓ Fetch
PC → 3059

$$\begin{aligned}\rightarrow PC_{\text{new}} &= PC_{\text{old}} + \text{offset} \\ &= 3059 - 15 \\ &= \underline{\underline{3044}}\end{aligned}$$

Examples on Instruction Execution in COA

❑ **GATE 2007 CS** – Consider the following program segment. Here R1, R2 and R3 are the general purpose registers.

Instruction	Operation	Instruction size (Words)
✓ MOV R1, (3000)	$R1 \leftarrow M[3000]$, $R_1 = 10$	2
→ LOOP: ✓ MOV R2, (R3)	$R2 \leftarrow M[R3]$, $R_2 = 100$	1
ADD R2, R1	$R2 \leftarrow R1 + R2$, $R_2 = 110$	1
✓ MOV (R3), R2	$M[R3] \leftarrow R2$	1
INC R3	$R3 \leftarrow R3 + 1$	1
DEC R1	$R1 \leftarrow R1 - 1$	1
BNZ LOOP	Branch on not zero	2
HLT	Stop	1

Memory

Add.	data
2000	→ 100, 110
⋮	⋮
<u>2010</u>	→ 100
3000	→ 10

$R_3 = 2000, 2001,$

❑ Assume that the content of memory location 3000 is 10 and the content of the register R3 is 2000. The content of each of the memory locations from 2000 to 2010 is 100. The program is loaded from the memory location 1000. All the numbers are in decimal. Assume that the memory is word addressable. The number of memory references for accessing the data in executing the program completely is:

A. 10 B. 11 C. 20 D. 21

$$\begin{aligned} & \text{loops} \times 2 + 1 \\ & = 10 \times 2 + 1 = 21 \end{aligned}$$

❑ **GATE 2007 CS** – Consider the data given in above question. Assume that the memory is word addressable. After the execution of this program, the content of memory location 2010 is:

A. 100 B. 101 C. 102 D. 110

	Instruction	Operation	Add	Instruction size (Words)
	MOV R1, (3000)	$R1 \leftarrow M[3000]$	1000	2
LOOP:	MOV R2, (R3)	$R2 \leftarrow M[R3]$	1008	1
	ADD R2, R1	$R2 \leftarrow R1 + R2$	1012	1
	MOV (R3), R2	$M[R3] \leftarrow R2$	1016	1
	INC R3	$R3 \leftarrow R3 + 1$	1020	1
→ ☆ Interrupt	DEC R1 ← PC	$R1 \leftarrow R1 - 1$	1024	1
	BNZ LOOP	Branch on not zero		2
	HLT	Stop		1

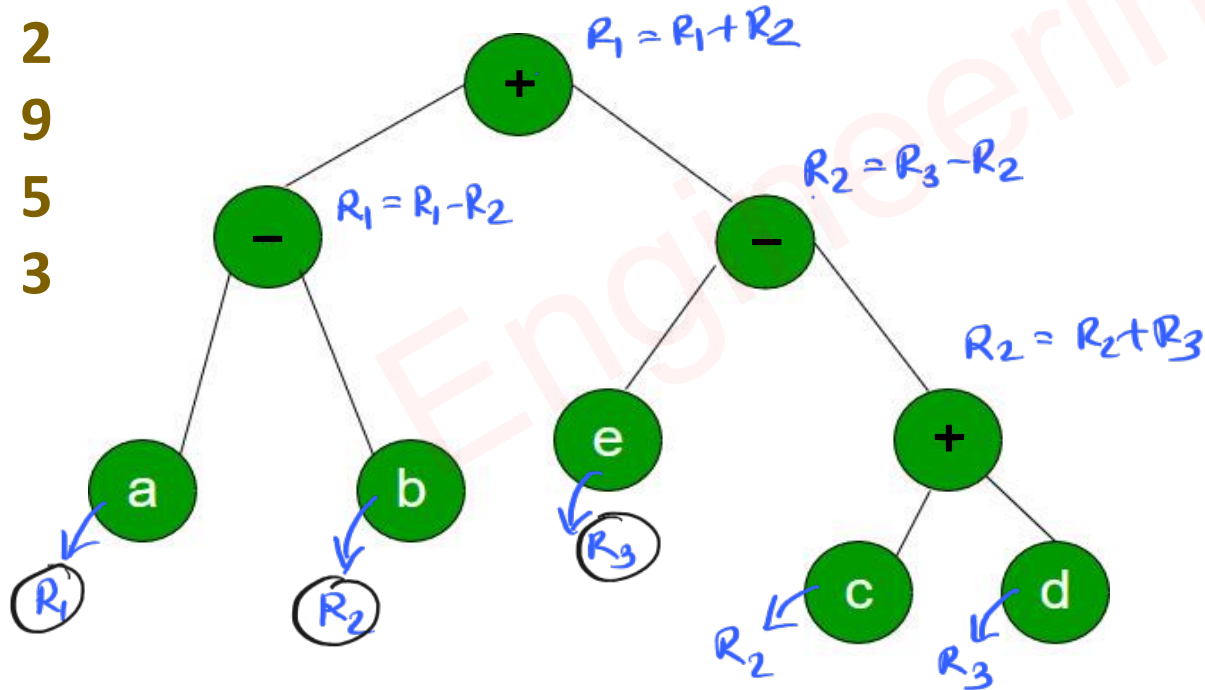
❑ **GATE 2007 CS** – Consider the data given in above questions. Assume that the memory is byte addressable and the word size is 32 bits. If an interrupt occurs during the execution of the instruction “INC R3”, what return address will be pushed on to the stack?

A. 1005 B. 1020 C. 1024 D. 1040

Examples on Instruction Execution in COA

❑ **GATE 2011 CS** – Consider evaluating the following expression tree on a machine with load-store architecture in which memory can be accessed only through load and store instructions. The variables a , b , c , d and e initially stored in memory. The binary operators used in this expression tree can be evaluated by the machine only when the operands are in registers. The instructions produce results only in a register. If no intermediate results can be stored in memory, what is the minimum number of registers needed to evaluate this expression?

- A. 2
- B. 9
- C. 5
- ✓ D. 3



❑ **GATE 2010 CS** – The program below uses six temporary variables a, b, c, d, e, f.

a = 1 → $R_1 = a$ X

b = 10 → $R_2 = b$ X

c = 20 → $R_3 = c$

d = a + b → $R_1 = a + b = d$ X

e = c + d → $R_1 = c + d = e$ X

f = c + e → $R_2 = c + e = f$

b = c + e → $R_1 = c + e = b$ X

e = b + f → $R_1 = b + f = e$ X

d = 5 + e → $R_1 = 5 + e = d$

return d + f → $R_1 = d + f$

Assuming that all operations take their operands from registers, what is the minimum number of registers needed to execute this program without spilling?

A. 2

✓ B. 3

C. 4

D. 6

Examples on Instruction Execution in COA

❑ **GATE CS** – In a simplified computer the instructions are:

OP Rj, Ri ; Performs Rj OP Ri and stores the result in register Ri

OP m, Ri ; Performs Value OP Ri and stores the result in Ri, where Value is data pointed by memory m

MOV m, Ri ; Moves the content of memory location m to register Ri

MOV Ri, m ; Moves the content of register Ri to memory location m

❑ The computer has only two registers and OP is either ADD or SUB. Consider the following basic block:

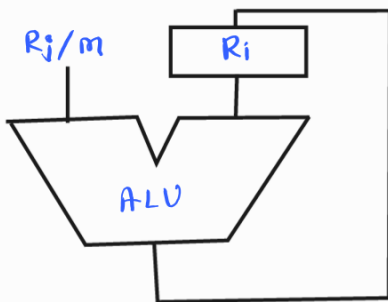
→ $t1 = a - b$ ① ✓

→ $t2 = c + d$ ① ✓

$t3 = e - t2$ ✗

$t4 = t1 - t3$ ✗ ① ✓

❑ Minimum number of MOV instructions required to perform above task. {All the operands are there in memory initially and stores the result in memory at last}



MOV mcb), Ri ; $R_i \leftarrow mcb)$

SUB mca), Ri ; $R_i \leftarrow a - b$, $R_j = t_1$

MOV mcd), Rj ; $R_j \leftarrow mcd)$

ADD mce), Rj ; $R_j \leftarrow c + d$, $R_j = t_2$

SUB m(e), Ri ; $R_j \leftarrow e - t_2$, $R_j = t_3$

SUB Ri, Rj ; $R_i \leftarrow t_1 - t_3$, $R_i = t_4$

MOV Rj, m(t4) ;

Examples on Instruction Formats in COA

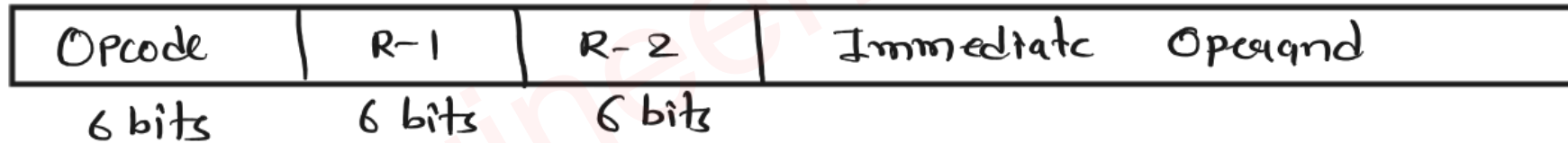
- ❑ **GATE 2014 CS** – A machine has a 32-bit architecture, with 1-word long instructions. It has 64 registers, each of which is 32 bits long. It needs to support 45 instructions, which have an immediate operand in addition to two register operands. Assuming that the immediate operand is an unsigned integer, the maximum value of the immediate operand is 16383.

- ✓ A. 16383
B. 16338
C. 16388
D. 16484

→ Size of Instruction = 32 bits

Registers = 64

Instructions = 45



→ Size of Immediate Operand = $32 - 6 - 6 - 6 = 14$ bits.

↓ Unsigned Integer
0 to $2^n - 1$

❑ **GATE 2005 CS** – Consider a three word machine instruction

ADD A[R0], @B

❑ The first operand (destination) "A [R0]" uses indexed addressing mode with R0 as the index register. The second operand (source) "@ B" uses indirect addressing mode. A and B are memory addresses residing at the second and the third words, respectively. The first word of the instruction specifies the opcode, the index register designation and the source and destination addressing modes. During execution of ADD instruction, the two operands are added and stored in the destination (first operand). The number of memory cycles needed during the execution cycle of the instruction is

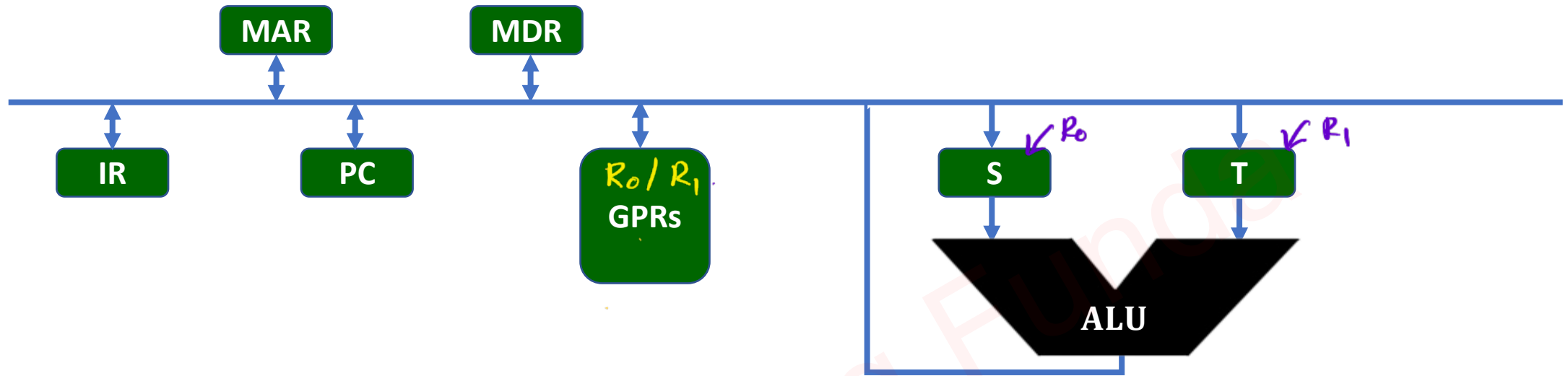
- A. 3
- B. 4
- C. 5
- D. 6

$$A[R_0] = [A + R_0]$$

$$① B = [B]$$

- 1 → Inst.ⁿ Fetch
- 2 → Operand 1 { A[R₀] }
- 3 → Operand 2 { @B }
- 4 → Write back { A[R₀] }.

❑ **GATE 2005 CS** – Consider the following data path of CPU.



❑ The ALU, the bus and all the registers in the data path are of identical size. All operations including incrementation of the PC and the GPRs are to be carried out in the ALU. Two clock cycles are needed for memory read operation - the first one for loading address in the MAR and the next one for loading data from the memory bus into the MDR. The instruction "add R0, R1" has the register transfer interpretation $R0 \leftarrow R0 + R1$. The minimum number of clock cycles needed for execution cycle of this instruction is.

- A. 2
- ✓ B. 3
- C. 4
- D. 5

$T_1 : S \leftarrow R_0$

$T_2 : T \leftarrow R_1$

$T_3 : R_0 \leftarrow S + T$

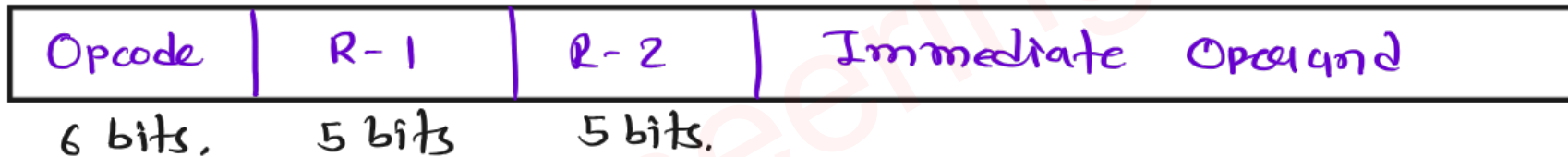
Examples on Instruction Formats in COA

- ❑ **GATE 2016 CS** – A processor has 40 distinct instructions and 24 general purpose registers. A 32-bit instruction word has an opcode, two register operands and an immediate operand. The number of bits available for the immediate operand field is _____

→ Instructions = 40

Registers = 24

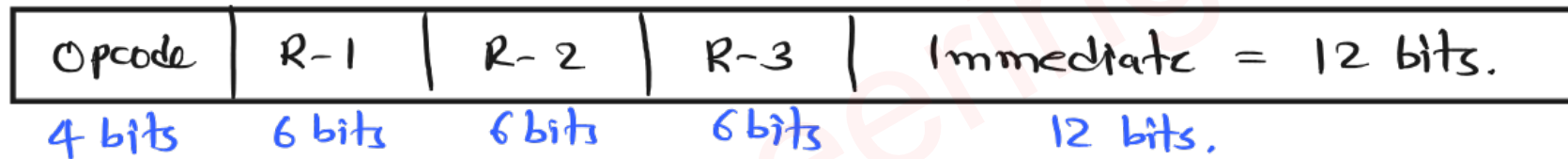
Inst.ⁿ Size = 32 bits.



→ Immediate Operand Size = $32 - 6 - 5 - 5$
= 16 bits.

- **GATE 2016 CS** – Consider a processor with 64 registers and an instruction set of size twelve. Each instruction has five distinct fields, namely, opcode, two source register identifiers, one destination register identifier, and a twelve-bit immediate value. Each instruction must be stored in memory in a byte-aligned fashion. If a program has 100 instructions, the amount of memory (in bytes) consumed by the program text is 500 byte.

→ Registers = 64
Instructions = 12



→ Inst.ⁿ size = $4 + 6 + 6 + 6 + 12 = \underline{\underline{34}}$ bits.

↓
4 byte 2 bits,

→ Inside memory we need 5 bytes for 34 bits with one instruction.

→ So for 100 Inst.ⁿ size of memory = $100 \times 5 = 500$ Bytes.

❑ **GATE 2008 CS** – A processor that has carry, overflow and sign flag bits as part of its program status word (PSW) performs addition of the following two 2's complement numbers 01001101 and 11101001. After the execution of this addition operation, the status of the carry, overflow and sign flags, respectively will be:

- A. 1, 1, 0
- ✓ B. 1, 0, 0
- C. 0, 1, 0
- D. 1, 0, 1

$$\begin{array}{r} \text{Carry} \rightarrow \begin{array}{c} 00 \\ 11 \end{array} \quad \begin{array}{ccccccc} & & 1 & & 1 & & \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \\ \hline \oplus & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \end{array} \\ \uparrow \\ \text{MSB sign } 0 \text{ +ve} \\ \quad \quad \quad 1 \text{ -ve} \end{array}$$

$$\text{Sign} = 0$$

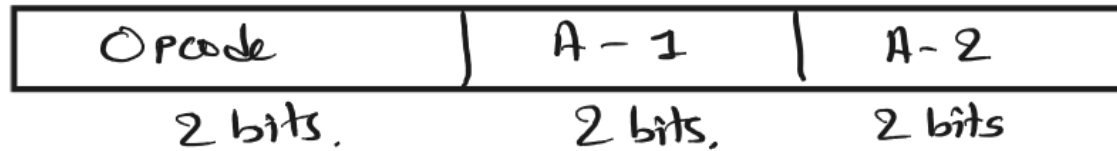
$$\text{Carry} = 1$$

$$\begin{aligned} \text{Overflow} &= C \oplus D_7 \\ &= 1 \oplus 1 \\ &= 0 \end{aligned}$$

Examples on Instruction Formats in COA

- ❑ **GATE CS** – Consider a system which supports 2 Address instructions and 1 Address instructions both. The system has 6 bit instructions and 2 bits addresses. If there are Two 2 address instructions in the system, then maximum and minimum how many 1 address instructions the system can support?

2- Add. Inst.ⁿ



↓
2 Inst.ⁿ

→ Total Encoding = 2 A.I.E. + 1 A.I.E

$$\Rightarrow 2^6 = 2 \times 2^4 + N \times 2^2$$

$$\Rightarrow 2^4 = 2^3 + N$$

$$\Rightarrow \boxed{N = 8}$$

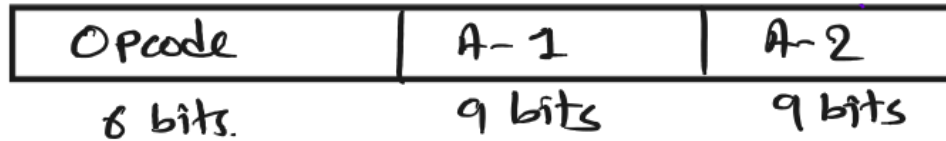
1 Add. Inst.ⁿ



↓
N Inst.ⁿ

- ❑ **GATE CS** – Consider a system with 24 bit instructions and 9 bit addresses. If there are Sixty 2 address instructions, then maximum how many 1 address instructions can be formulated in the system?

2 Add. Inst.ⁿ



↓
60 Inst.ⁿ

1 Add Inst.ⁿ



↓
N Inst.ⁿ

→ Total Encoding = 1-A.I.E + 2-A.I.E

$$\Rightarrow 2^{24} = N \times 2^9 + 60 \times 2^{18}$$

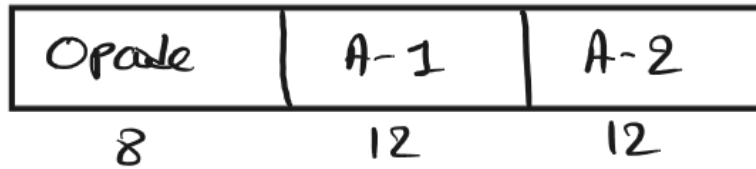
$$\Rightarrow 2^{15} = N + 60 \times 2^9$$

$$\Rightarrow N = 2^{15} - 60 \times 2^9$$

$$\Rightarrow \boxed{N = 2048}$$

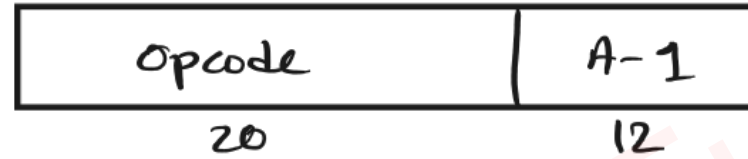
- ❑ **GATE CS** – Consider a system with 32 bit instructions and 12 bit addresses. If there are 250 2 address instructions, 24500 1 address instructions then maximum how many 0 address instructions can be formulated in the system?

2 - Add. Inst.ⁿ



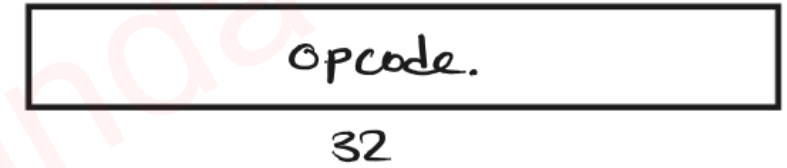
↓
250 Inst.ⁿ

1 - Add Inst.ⁿ



↓
24500 Inst.ⁿ

0 - Add. Inst.ⁿ



↓
N Inst.ⁿ

$$\Rightarrow \text{Total Encoding} = 2\text{-A.I.E} + 1\text{-A.I.E} + 0\text{-A.I.E}$$

$$\Rightarrow 2^{32} = 250 \times 2^{24} + 24500 \times 2^{12} + N$$

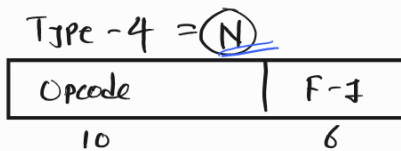
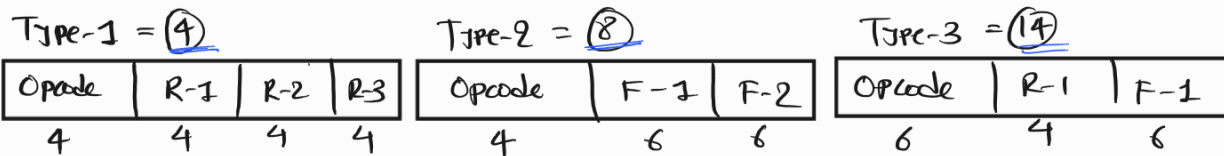
$$\Rightarrow N = 2^{32} - 250 \times 2^{24} - 24500 \times 2^{12}$$

$$\Rightarrow \boxed{N = 311296}$$

Examples on Instruction Formats in COA

GATE 2018 CS – A processor has 16 integer registers (R0, R1, ... , R15) and 64 floating point registers (F0, F1, ... , F63). It uses a 2-byte instruction format. There are four categories of instructions: Type-1, Type-2, Type-3, and Type 4. Type-1 category consists of four instructions, each with 3 integer register operands (3Rs). Type-2 category consists of eight instructions, each with 2 floating point register operands (2Fs). Type-3 category consists of fourteen instructions, each with one integer register operand and one floating point register operand (1R+1F). Type-4 category consists of N instructions, each with a floating point register operand (1F). The maximum value of N is 32 .

→ Registers = 16
Floating = 64
Instⁿ Size = 16 bits



→ Total Encoding = T1E + T2E + T3E + T4E

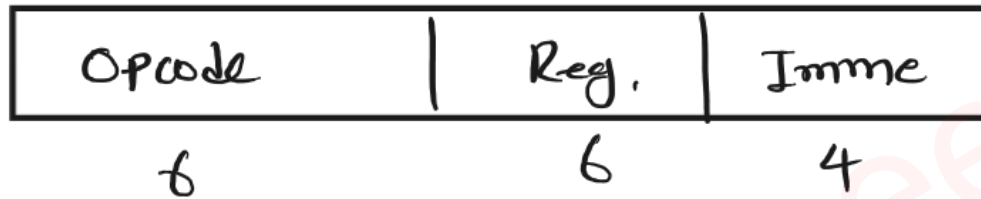
$$\Rightarrow 2^{16} = 4 \times 2^{12} + 8 \times 2^{12} + 14 \times 2^{10} + N \times 2^6$$
$$\Rightarrow 2^{10} = 4 \times 2^6 + 8 \times 2^6 + 14 \times 2^4 + N$$
$$\Rightarrow \boxed{N = 32}$$

□ **GATE 2020 CS** – A processor has 64 registers and uses 16-bit instruction format. It has two types of instructions: I-type and R-type. Each I-type instruction contains an opcode, a register name, and a 4-bit immediate value. Each R-type instruction contains an opcode and two register names. If there are 8 distinct I-type opcodes, then the maximum number of distinct R-type opcodes is 14

- ✓ A. 14
B. 15
C. 16
D. 12

→ Registers = 64
Inst.ⁿ Size = 16 bits

I-Type = (8)



R-Type = (N)



→ Total Encoding = I-T-E + R-T-E.

$$\Rightarrow 2^{16} = 8 \times 2^6 + N \times 2^{12}$$

$$\Rightarrow 2^4 = 2^1 + N$$

$$\Rightarrow \boxed{N = 14}$$